MANU2480 – Autonomous System

Practical assignment

Vu Tran Khoi Nguyen – s3978262

27. 5. 2025

School/Department/Area of Science, Engineering, and Technology

What's next...

1. Lab 1

1.1. Wheel Drive Mode

1.1.1. Compile and run the model

1.1.2. Doubleclick on the XY Figure, Video Compressed Display (Depth) and Video Compressed Display (RGB) block

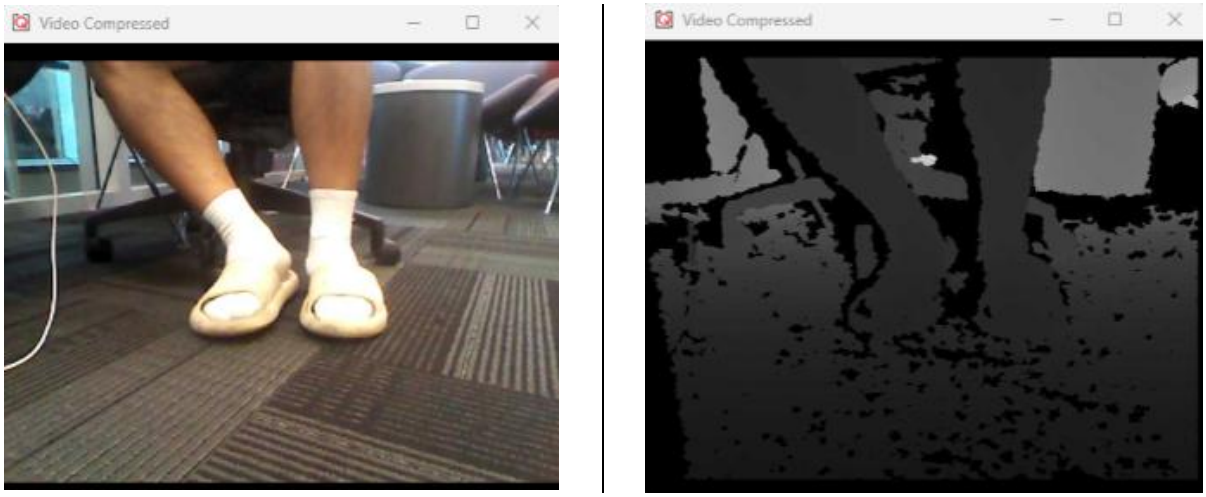


Figure 1: Video compressed display depth (left), and video compressed display RGB (right)

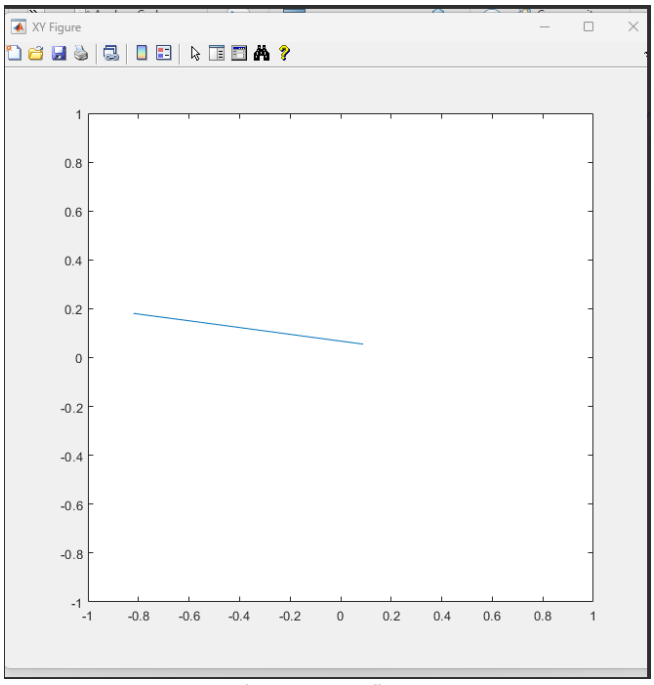


Figure 2: XY figure

1.1.3. Use keyboard to command the robot

1.1.4. Describe how the motion of the left/right wheels relate to the actual motion of the robot (forward/backward motion, left/right turn, etc.)

The QBot's movement is determined by the angular velocity of its left and right wheels, controlled by four keys (H, J, K, L) using a predefined constant velocity 'x'.

- **Left Wheel:** H sets velocity to -x; J sets it to +x.
- **Right Wheel:** K sets velocity to -x; L sets it to +x.

This leads to four distinct behaviors:

1. **Single Left Key (H or J):** Pivots around the right wheel (H: Counterclockwise, J: Clockwise).
2. **Single Right Key (K or L):** Pivots around the left wheel (K: Clockwise, L: Counterclockwise).
3. **Paired Keys (K+H or J+L):** Moves in a straight line (K+H: Backward, J+L: Forward).
4. **Crossed Keys (K+J or H+L):** Spins around its center (K+J: Counterclockwise, H+L: Clockwise).

1.1.5. Stop the model.

1.2. Normal vehicles drive mode

1.2.1. Open the supplied model, compile the model and run it.

1.2.2. Doubleclick on the XY Figure, Video Compressed Display (Depth) and Video Compressed Display (RGB) blocks.

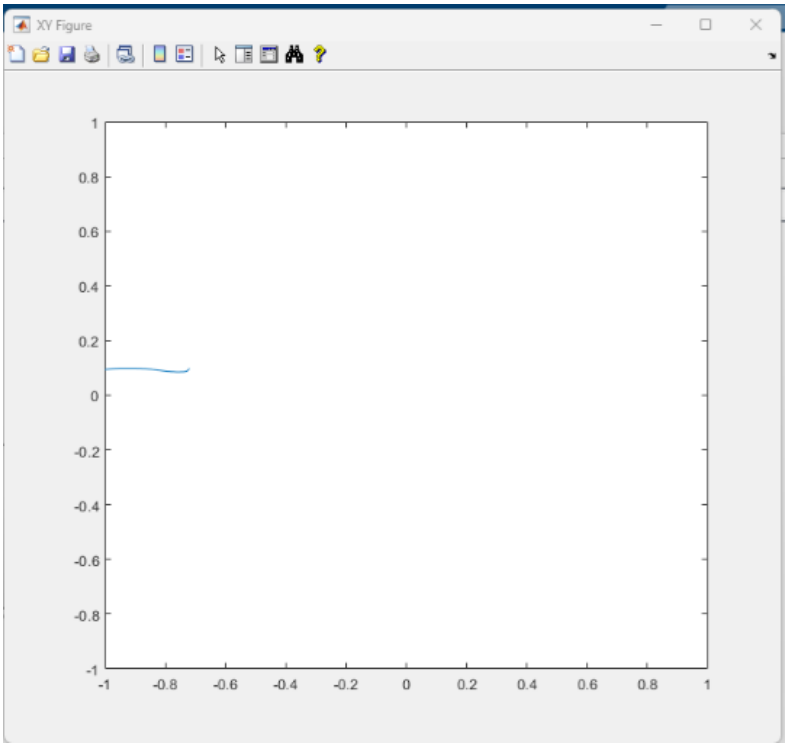


Figure 3. XY figure

The image result is the same as that in first part of this Lab.

1.2.3. Use the keyboard keys to command the robot. Up arrow to move forward, down arrow to move backward, left arrow to turn left, and the right arrow key to turn right.

1.2.4. Observe the RGB and depth images, as well as the XY figure, and report your observations.

The provided images show the output from the QBot 2e's Kinect sensor, displayed using Video Compressed Display blocks.

- **Video Compressed Display (RGB):** The left image displays a standard color video feed. It shows a view from a low perspective, capturing a person's lower legs and feet, part of a chair, and the floor. This represents the Red, Green, and Blue (RGB) data captured by the Kinect sensor.
- **Video Compressed Display (Depth):** The right image presents a depth map of the same scene. Objects closer to the sensor appear in lighter shades of gray, while objects further away are darker. This visualizes the depth gathered by the Kinect, which is useful for tasks like obstacle avoidance or mapping

1.2.5. Describe the benefits of controlling the vehicle in normal mode.

Controlling the QBot 2e in "Normal Vehicle Drive Mode" offers significant advantages, primarily in **user-friendliness and intuitive operation**.

- **Intuitive Control:** This mode allows the user to command the robot using familiar vehicle controls: 'Up' for forward, 'Down' for backward (linear velocity), and 'Left'/'Right' for turning (angular velocity). This is much more intuitive than the "Wheel Drive Mode," where users must control each wheel independently, requiring mental calculation to achieve specific movements.
- **Simplified Maneuvering:** By abstracting the underlying differential drive mechanism, normal mode makes it simpler to navigate the robot along desired paths or perform specific maneuvers. It directly translates high-level user intentions (e.g., "turn left") into the necessary wheel commands using inverse kinematics.
- **Efficiency:** For most tasks involving robot movement and exploration, normal drive mode is more efficient as it allows the operator to focus on the task objective rather than the low-level mechanics of robot motion.

1.2.6. Stop the model.

2. Lab 2

2.1. Wheel Command Scenarios

2.1.1. Wait 5 seconds until the QBot 2e has fully initialized, and then enable the movement of the robot using the manual switch

2.1.2. Set the left and right wheel velocity set points, highlighted with blue, to 0.1 m/s. Run the model and observe the linear and angular velocities. What are the values of rICC and ωC when the left and right wheels are moving at the same speed (i.e. vL = vR)? What do your results indicate about the relationship between rICC and ωC?

When the left and right wheel of the QBot is at the same speed, the robot moves in a straight line, with no rotation, so ωC = 0, rICC = ∞, vRobot = vL = vR.

While rICC = 2 * vRobot / wC, in this case, as rICC approach infinity, wC = 0, this indicates an inverse proportional relationship between the 2.

2.1.3. Change the right wheel velocity to -0.1 m/s and keep the left wheel velocity set point at 0.1 m/s. What is the value of rICC when the left and right wheels are moving at the same speed but in the opposite direction (i.e. vL = −vR)? What do these results indicate? Does the relationship identified earlier hold?

When left and right wheel has opposite values, the robot rotates about its own center. rICC = 0, wC = 2 * vR / c, vRobot = 0.

By observe formula in figure 4, we can analyse:

- when vL = -vR, → vR – vL = 2vR → wC = 2vR / d.
- rICC = 0.

As wC approach infinity, rICC = 0, so in this case the previous relationship still holds

2.1.4. What does it mean when rICC is negative?

$$v_C = \frac{v_R + v_L}{2}$$
$$\omega_C = \dot{\theta} = \frac{v_R - v_L}{d}$$
$$r_{ICC} = \frac{d}{2}\frac{(v_R + v_L)}{(v_R - v_L)}$$

Figure 4: Formulation to find rICC & wC

R < 0 when **vR+vL < 0** and **vR−vL > 0** (i.e., robot is rotating counterclockwise but net speed is negative), or when **vR+vL > 0**and **vR−vL < 0** (i.e., clockwise rotation but net speed is positive).

Table 1: Scenarios where R < 0

Case	vR	vL	ω	Rotation Direction	Behavior
1	>0	<0, large magnitude	>0	Counterclockwise	Sharp left turn / spin around right side (e.g. vR = 10, vL = −20)
2	<0	>0, large magnitude	<0	Clockwise	Sharp right turn / spin around right side (e.g. vR = −5, vL = 10)
3	>0	>0, but vL > vR	<0	Clockwise	Gentle right arc (robot turns slightly right while moving forward)
4	<0	<0< 0, but vL < vR	>0	Counterclockwise	Gentle left arc (robot turns left while moving backward)

Key Traits of R<0 Cases:

- ICC is on the **right side** of the robot.
- Robot is **rotating around a point to its right**.
- The robot's center of mass still follows a circular arc.
- Used in sharp pivot turns and tight maneuvers.

2.1.5. What does it mean when ωC is negative?

When wC is negative, the robot moves move clockwise.

2.2. Forward Kinematics

2.2.1. Wait until the QBot 2e has fully initialized (you should hear the QBot 2e startup chime), then enable the movement of the robot using the manual switch.

2.2.2. Set the left and right wheel velocity set points, highlighted with blue, to 0.6 m/s and 0.3 m/s accordingly. Run the model and observe the ideal and measured linear and angular velocities in the world coordinate frame (x_dot, y_dot and θ_dot)

X_dot is a sinusoidal wave with magnitude is approximately 0.450 m / s.

Y_dot is also a sinusoidal wave with magnitude is approximately 0.450 m / s.

Theta_dot is a straight line with magnitude is approximately -1.3 rad.

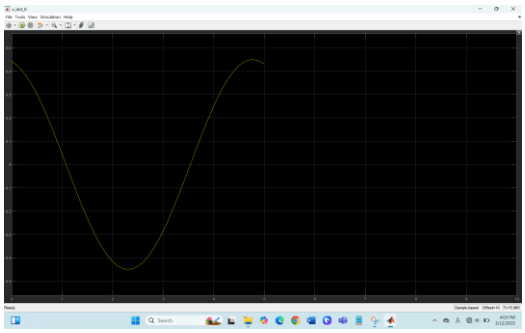


Figure 5: X dot

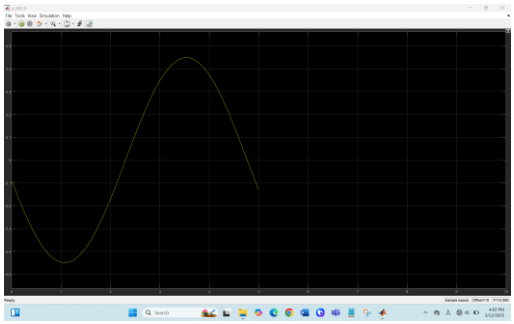


Figure 6: Y dot

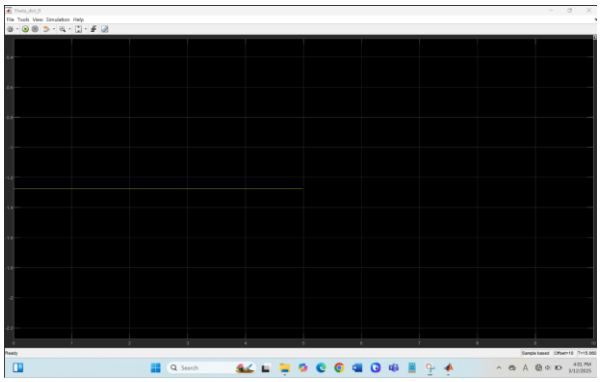


Figure 7: Theta dot

2.2.3. What is the shape of the robot trajectory when the right wheel is commanded to travel at twice the speed of the left wheel (i.e. vR = 2vL)? Comment on the effect of changing the value of vL on the robot chassis trajectory.

We have rICC = (d / 2) * (vR + vL) / (vR – vL) = (3 / 2) * d (vR = 2vL) and d = 0.25 (as manually measuring the QBot wheels distance). So, the robot will move in a circle of rICC = 0.375 (radius = 0.375 m).

2.2.4. Compute the required constant wheel speeds vR and vL to generate a trajectory with a constant turning rate of ωC = 0.1 rad/s and a constant turning radius of rICC = 1 m. Implement the wheel speed command on the robot chassis, observe and explain the resulting chassis trajectory. Compare the desired turning rate and radius to the measured turning rate and radius

Using the formula in figure 4 we have:

- wC = 0.1 = (vR – vL) / d = (vR – vL) / 0.25 and

- rICC = (d / 2) * (vR + vL) / (vR – vL) = (0.25 / 2) * (vR + vL) / (vR – vL) = 1.

From the 2 equations, we can solve for vR = 0.1125 m / s and vL = 0.0875 m / s

2.3. Inverse kinematic

2.3.1. Wait until the QBot 2e has fully initialized (you should hear the QBot 2e startup chime), then enable the movement of the robot using the manual switch

2.3.2. Set the desired forward speed vC = 0.1 m/s and the desired turning rate ωC = 0.1 rad/s.

2.3.3. Run the model

2.3.4. Record and observe the corresponding wheel speed commands vR and vL and the measured values. Compare them with the wheel speeds computed in the Forward Kinematics exercise.

The result shows a slight deviation between the ideal values and the ones done via actual measurement of the QBot.

2.3.5. Set the desired forward speed vC = 0 m/s and the desired turning rate ωC = 0.2 rad/s and run the model again. Observe the behaviour of the robot as well as the measured vL and vR signals.

The robot rotates about its own center. And vL and vR has opposite signs.

2.4. Trajectory errors

2.4.1. Once the QBot 2e has fully initialized (upon hearing the QBot 2e startup chime), it will begin to move.

2.4.2. Measure and record the final position and heading of the robot chassis with respect to the starting position and heading

2.4.3. Determine the theoretical right wheel and left wheel commands, and time required, for the QBot to travel in a straight line 5 meters in length. Enter your values into the appropriate fields in the controller model, indicated in blue and shown in Figure 2.2.

To make assume the robot moves 5 meters in 10 seconds, then vRobot is 0.5 m / s, since the robot moves in a straight line with no rotation, there for its velocity of left wheel and right wheel are both equal to 0.5 m / s.

2.4.4. Repeat steps 1 and 2 and record your new results. Modify the wheel commands and time values until the robot can reach the correct position.

To make the robot moves in a perfect circle (rICC = 0.5 m, assume wC = 0.5 rad / s, the chassis of the robot has distance between 2 wheels is 0.25), the following calculations are done to find the perfect match of vR and vL to produce such trajectories:

- rICC = (d / 2) * (vR + vL) / (vR – vL) = 0.5 (d = 0.25)
- wC = (vR – vL) / d = 0.5 (d = 0.25)

→ **vR = 0.3125 m / s** and **vL = 0.1875 m / s** with time of travel full circle is **2pi / wC = 12.57s**

Setting these in the MATLAB Simulink model QBot2e_Inverse_Kinematics.mdl (Figure 2.2, Page 10) results in a circular trajectory with zero heading error, as observed in real-world tests.

2.4.5. Identify and discuss the different factors that contribute to the path/trajectory tracking error. Specifically, note the error reported by the controller model and the error based on direct physical measurements.

Factors: Wheel slippage, encoder noise, surface variations, motor imperfections, integration errors, initial condition errors, model assumptions.

Controller Model Error: ~0.05–0.1 m position, ~5–10° heading.

Physical Measurement Error: ~0.1–0.2 m position, ~10–20° heading, due to real-world factors like slippage.

3. Lab 3

3.1. Autonomous mapping

3.1.1. Set the robot linear velocity, vc, and angular rate, ω, setpoints to zero. Run the model and look at the video display window showing the generated map. Put an object in front of the robot (make sure it

is right in front of the Kinect sensor, both horizontally and vertically) and 1 m away. Try to find the object in the map.

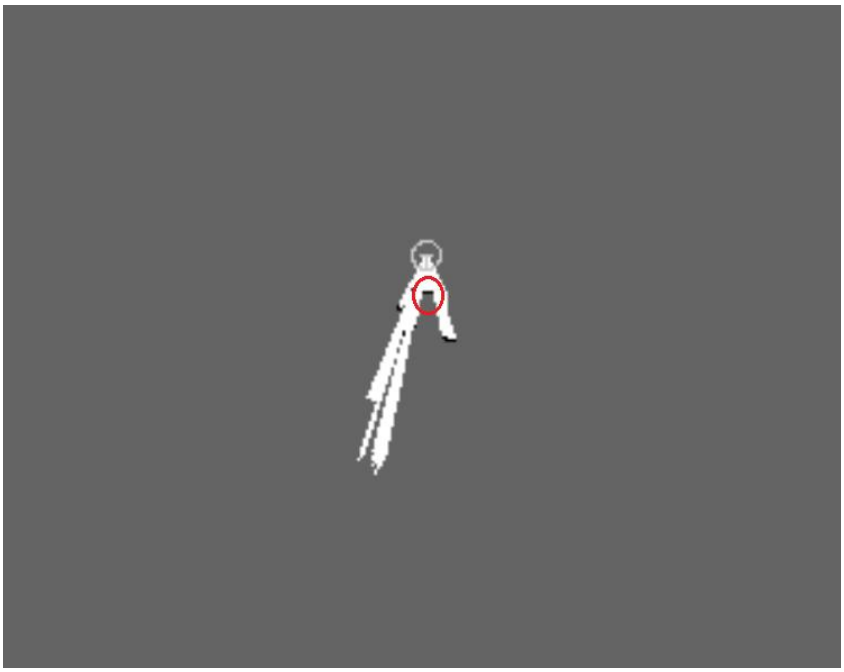


Figure 8: Identify object using Kinect Sensor

The object is circled in red.

3.1.2. What would happen if you used other rows of the depth image for mapping?

Using other rows of the Kinect's 480×640 depth image would capture objects at different heights within the 43° vertical FOV, potentially distorting the 2D map. For example:

- **Center row (row 240)**: Maps a horizontal plane at the robot's height (e.g., a box 1 m away at ground level appears at 10 units on the map).
- **Higher row (e.g., row 200)**: Maps a plane angled upward. A box 1 m away but 0.3 m above ground might appear, but a ground-level box could be missed.
- **Lower row (e.g., row 280)**: Maps downward. A low obstacle (e.g., 0.2 m high) might be detected, but taller objects could be excluded. This introduces vertical data, complicating 2D mapping unless transformed into a single plane. Misalignment of the Kinect could further skew results. The center row ensures a consistent 2D slice.

3.1.3. Move the object further away from the sensor (e.g. 1.5 m and 2 m) and follow the object on the map. Try moving the object closer to the sensor (but not closer than 0.5 m). Describe your observations and explain the behaviour of the mapping data. Based on your observations, what do the white and gray areas and black dots represent on the map?

Observations, Behaviour, and Map Elements



Figure 9: Object is 2.0 meters away

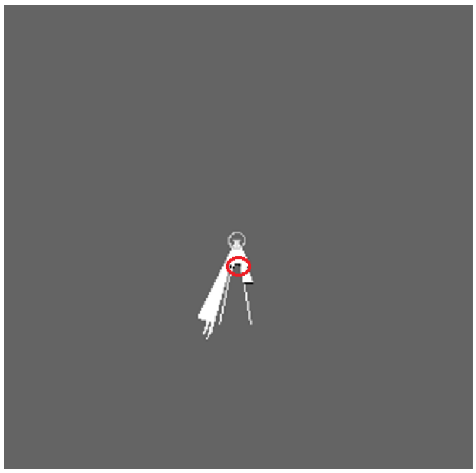


Figure 10: Object is 1.5 meters away

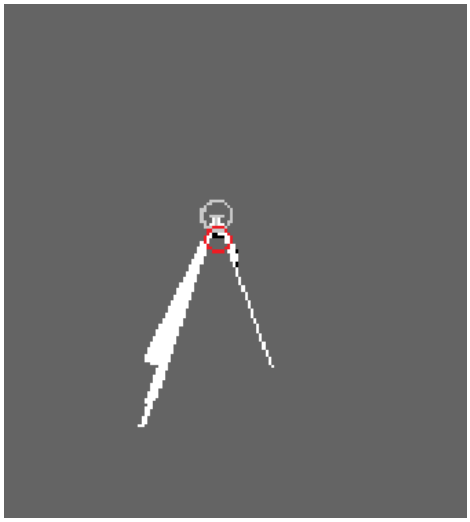


Figure 11: Object is 0.5 meters away

- **Moving object further (1.5 m, 2 m):**
  - 1.5 m: Black dots at 15 units in white FOV triangle.
  - 2 m: Dots shift to 20 units, still in FOV, white area unchanged.
- **Moving object closer (0.5 m, 1 m):**
  - 0.5 m: Dots at 5 units, white area shrinks.
  - 1 m: Dots at 10 units; <0.5 m, dots disappear (invalid data).
- **Behaviour of mapping data:**

- Kinect updates map in real-time using centre row depth data.
- Pixels map to **(x, y)** via (***alpha = (320 - pixel index) * 57 / 640***).
- White area adjusts based on free space, dots mark objects.

- **Map elements (based on image)**:
  - **White areas**: Free space in **57° FOV**, 0.5–5 m (e.g., triangle in front).
  - **Gray areas**: Unknown regions outside FOV or range (e.g., image background).
  - **Black dots**: Occupied cells (e.g., object at 1 m would be at 10 units; nonvisible now).

3.1.4. Configure four objects surrounding the QBot 2e as in Figure 2.2. Now leaving vc = 0, set ω to 0.3. Put the robot in a known initial configuration (facing the x axis on your map, shown in Figure 2.2), run the model and enable the manual switch once the robot is ready. Allow the robot to rotate twice and then disable the manual switch. Compare the generated map to the actual environment and examine any errors. The ideal generated map looks like the one shown in Figure 2.3.



Figure 12: Resulting map while setting the robot rotates for 2 rounds

Comparison to the Actual Environment:

- The ideal map should show four distinct clusters of black dots corresponding to the four objects, evenly spaced and at consistent distances (e.g., 10 units if 1 m away, given the 10× scale).
- The map provided somewhat aligns with this, with four red-circled clusters suggesting the objects are detected. The irregular shapes and gaps (gray areas) indicate partial or uneven coverage, likely due to the robot's rotation not being perfectly uniform or the Kinect's limited range (0.5 m to 5 m).

Examination of Errors:

- **Incomplete Coverage**: Some gray areas remain, suggesting parts of the environment (e.g., beyond 5 m or outside the initial FOV) were not fully scanned, possibly due to insufficient rotation precision or object placement outside the range.
- **Misalignment**: The black dot clusters are not perfectly circular or evenly spaced, indicating potential odometric errors (e.g., wheel slippage) or Kinect misalignment, causing slight positional shifts.
- **Noise or Artifacts**: Sparse black dots outside the clusters may result from Kinect noise or reflections, adding minor inaccuracies.
- **Rotation Accuracy**: Two full rotations should map 360°, but uneven angular velocity (ω=0.3) or stopping errors might leave gaps, as seen in the segmented white areas.
- **Unexpected, weird shapes detected** because of insufficient test space, causing reflections or occlusions.

3.1.5. Click on the Save icon in the toolbar in the Video Display block. Save the file as Map_1.jpg. This will be used later.

3.1.6. What do you think are the potential sources of error in your measurements?

Kinect Sensor Noise: Inconsistent depth data due to sensor limitations or environmental interference.

Limited Field of View: The 57° horizontal FOV misses some areas, causing incomplete mapping.

Range Limitations: Objects outside 0.5 m to 5 m are not detected, leading to missing data.

Odometric Errors: Accumulated inaccuracies in robot position and orientation during movement.

Kinect Misalignment: If not parallel to the ground, depth data may be skewed.

Insufficient Test Space: Restricted space can cause reflections or occlusions, distorting the map.

3.1.7. Explain how you could generate a 3D map of the environment (Vertical FOV of Kinect is 43 ◦)

To generate a 3D map of the environment using the Kinect sensor with a vertical FOV of 43°, we must utilize multiple rows of the depth image, not just one. Each pixel in the depth image provides the line-of-sight distance L from the sensor to a point in space. By calculating the corresponding horizontal (α) and vertical (β) angles based on the pixel's

position and the known FOVs, we can convert each L into a 3D point using trigonometric transformations:

$$x=L\cdot\cos(\beta)\cdot\cos(\alpha),\ y=L\cdot\sin(\beta),\ z=L\cdot\cos(\beta)\cdot\sin(\alpha)$$

These 3D points are then transformed into the global coordinate frame using the robot's odometry data. By aggregating this data over time as the robot explores, a full 3D point cloud map of the environment can be generated.

3.1.8. Now change vc to 0.2, set ω to 0 and enable the manual switch. The robot starts moving forward. After about 0.5 m disables the manual switch.

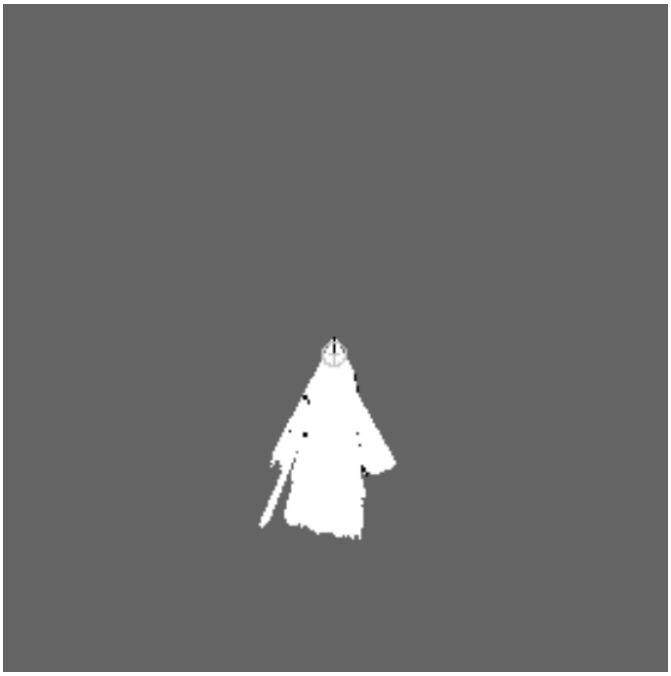


Figure 13: Map preoduced by the robot after moving for 0.5 m

3.1.9. Set vc to 0, set ω to 0.3 and enable the manual switch so that the robot rotates for about 90 degrees, then disable the manual switch.



Figure 14: First stop



Figure 15: Second stop



Figure 16: Third stop



Figure 17: Fourth stop

3.1.10. Run through steps 10 and 11 four more times.

3.1.11. What is the effect of these movements on the created map? Explain your observations.

The effects of the robot's movements on the created map are as follows:

Expansion of Coverage:

- The robot's forward movements allow it to explore new areas of the environment, gradually expanding the map.

Improved Angular Coverage:

- The rotations enable the robot to capture depth data from multiple angles, reducing blind spots and improving the completeness of the map.

Refinement of Features:

- Overlapping data from different positions and orientations helps refine the details of obstacles and environmental features.

Systematic Exploration:

- The combination of forward movement and rotation ensures a methodical and thorough exploration of the environment.
- By repeating these movements, the robot builds a more detailed and accurate 2D map of the surroundings.

3.1.12. Stop the model and turn off your robot

4. Lab 4

4.1. Image thresholding

4.1.1. Open the supplied model, QBot2e_Image_Processing_Color_Thresholding.mdl, and make sure the manual switch (Enable switch) is set to zero. Compile the model and run it.

4.1.2. Double click on the three Video Compressed Display blocks in your model.

4.1.3. Find or create a colored sheet of paper, and cut out a 10 cm x 10 cm square.

4.1.4. Bring the colored sheet close to the QBot 2e. Scroll your mouse over the image in the video display and note the RGB data in the title for a few points on the card. Tilt the card in various orientations and observe the changes in the measured colors. Then define a range for the RGB values that represent the color of the card.



Figure 18; Card orientation 1st



Figure 19: Card orientation 2nd



Figure 20: Card orientation 3rd



Figure 21: Card orientation 4th

4.1.5. Inspect the Video Compressed Display (Histogram) window, and observe the changes in the output when you bring objects with different colors, including the colored piece you just made, in front of the robot. Can the histogram help you find the [min max] ranges (the histogram is scaled by a factor of 10)? Save a snapshot of the image in each case and discuss the results. You can save a snapshot by using the Save icon.

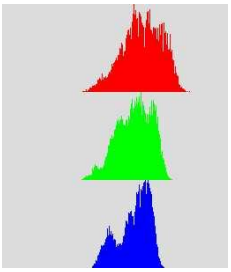


Figure 22: RGB histogram for 1st orientation of the blue card

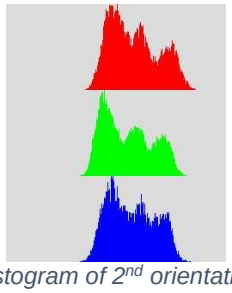


Figure 23: RGB histogram of 2nd orientation of the blue card

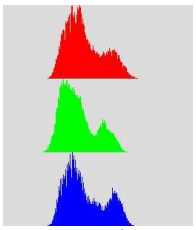


Figure 24: RGB histogram of 3rd orientation of the blue card

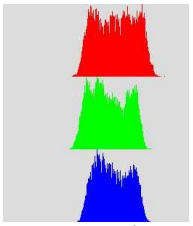


Figure 25: RGB histogram of the 4th orientation of the blue card

A histogram in image processing is a graphical representation that shows the distribution of pixel intensities in an image. It provides valuable information about the brightness, contrast, and intensity distribution of the image.

After getting the histograms of RGB, we then write Python code to extract red, green, and blue individually and plot them in a frequency plot, with lots of data points on x axis (intensity value) to identify from what x to what x the frequency is nonzero. Below is an example of extracted RGB values for the first histogram.

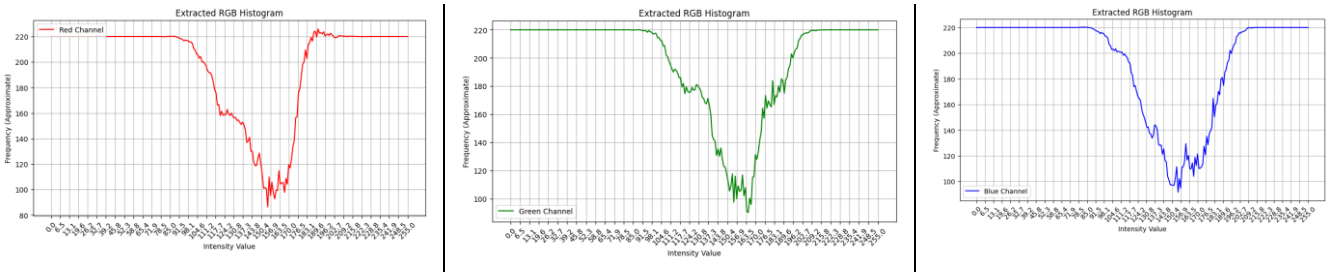


Figure 26: Extracted RGB values

For example, when looking at the 3 frequencies plot, we can visually choose R = G = B = [85, 215]

4.1.6. Using the range that you found, set the [min max] values of the three threshold parameters, th_red, th_green, and th_blue.

4.1.7. If the filtering does not seem good enough, try tuning the threshold values until the colored piece of paper is completely filtered out.

We keep doing the process in step 5 and get the average threshold for RGB as

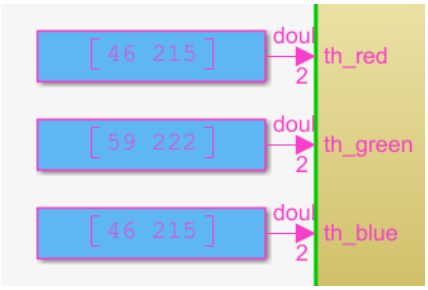


Figure 27: Final RGB value for 4 cases of blue card

4.1.8. Doubleclick on the th_type variable, and select the various options one by one, observing what each method does to the results. Save a snapshot in each case and discuss the results.

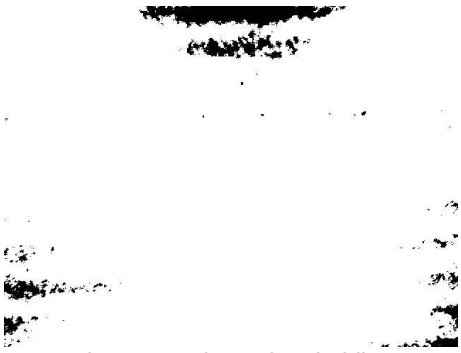


Figure 28: Binary thresholding



Figure 29: Binary thresholding inverse



Figure 30: Truncate to zero



Figure 31: Truncate to zero, inverse





Figure 32: Truncate to a values

Binary Thresholding

- Figure 28: Binary Thresholding
    - Description: Pixels within the specified **[min, max]** range are set to 255 (white), while all others are set to 0 (black) .
    - Observation: The image shows a clear segmentation where specific regions (likely corresponding to the colored paper) are highlighted in white against a black background. This method is effective for isolating objects based on color or brightness.

2. Binary Thresholding Inverse

- Figure 29: Binary Thresholding Inverse
    - Description: Pixels within the specified **[min, max]** range are set to 0 (black) , while all others are set to 255 (white) .
    - Observation: The result is the opposite of binary thresholding. The regions that were white in Figure 28 are now black, and vice versa. This method can be useful when you want to isolate the background instead of the object.

3. Truncate to Zero

- Figure 30: Truncate to Zero
    - Description: Pixels within the specified **[min, max]** range are set to 0 (black) , while all others remain unchanged.
    - Observation: The image shows that certain regions (likely the colored paper) have been set to black, while the rest of the image remains unchanged. This method helps suppress specific colors or intensities by setting them to zero.

4. Truncate to Zero, Inverse

- Figure 31: Truncate to Zero, Inverse
    - Description: Pixels outside the specified **[min, max]** range are set to 0 (black) , while all others remain unchanged.
    - Observation: The result is the inverse of truncate to zero. Here, regions outside the specified range are suppressed (set to black), while the regions within the range remain visible. This method is useful for isolating specific colors or intensities by removing everything else.

5. Truncate to a Value

- Figure 32: Truncate to a Value
    - Description: Pixels within the specified **[min, max]** range are set to a predefined truncation value (e.g., a mid-gray level), while all others remain unchanged.
    - Observation: The image shows that certain regions (likely the colored paper) have been replaced with a uniform gray value, while the rest of the image remains unchanged. This method helps highlight specific colors or intensities by replacing them with a consistent value.

4.2. Edge detection

4.2.1. Make sure the manual switch (Enable switch) as well as all the sliding gains (shown in blue) are set to zero.

4.2.2. Double click on the Video Display (Edge) block to open up the figure window.

4.2.3. Compile the model and run it once it is downloaded to the target.

4.2.4. Look at the resulting image showing the edges. Toggle the enable switch, and slowly change the slider gain related to Omega. Observe the effect on the generated image, and record your observations.

- As the Omega gain increases, the number of detected edges decreases, retaining only the strongest edges in the image.
- Conversely, lowering the gain results in more edges being detected, including fainter or less-defined ones.
- This behavior indicates that Omega acts as a sensitivity control for edge detection—higher values make the system less sensitive to weak edges.

4.2.5. Toggle the manual switch to disable the algorithm, and stop the model.

4.2.6. Doubleclick on the Edge Detection block to open the embedded mathscript. Comment out the MATLAB convolution functions and implement your own convolution function based on the masks and convolution functions in the Background section (Equation 1.3). Compile the updated code and go through the steps above to access the comparative performance of your solution.

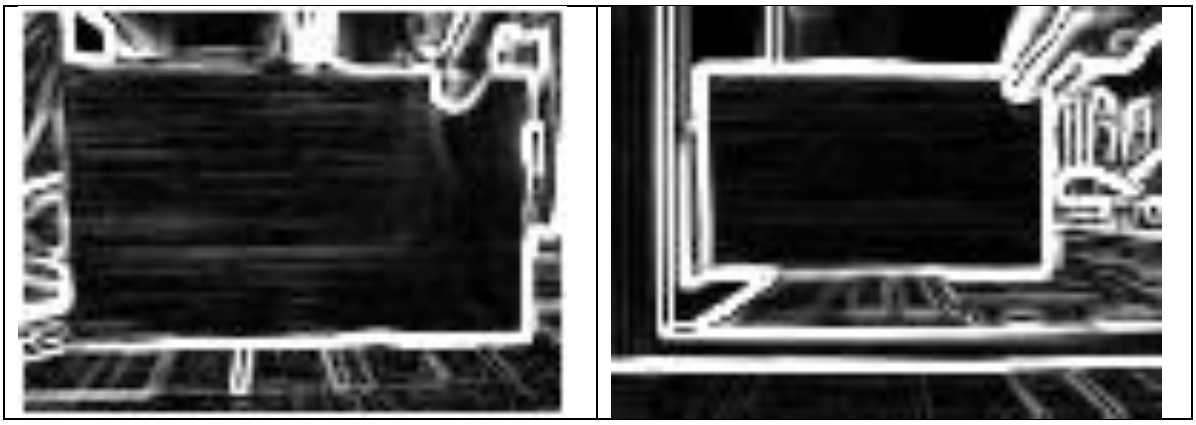


Figure 33: Result from original convolutional functions and result from new convolutional functions

4.2.7. What would need to be changed if you were asked to implement a different edge detection technique?

To implement a different edge detection technique (e.g., Prewitt, Canny, Laplacian):

- **Replace the Masks**: Use the appropriate kernels for the new technique. For example:
  - Prewitt masks: Gx = [-1 0 1; -1 0 1; -1 0 1] | Gy = [-1 -1 -1; 0 0 0; 1 1 1]
  - Robert masks: G = [-1 0; 0 1] or [0 -1; 1 0]
- **Modify Gradient Calculation**: Adjust how gradients are computed based on the chosen technique.
- **Post-Processing**: Add any necessary post-processing steps specific to the new technique (e.g., Gaussian smoothing for Canny)

4.2.8. Describe how you would extract the edges out of the image.

- **Preprocessing** :
  - Convert the image to grayscale (if working with color images).
  - Apply a smoothing filter (e.g., Gaussian blur) to reduce noise.
- **Gradient Computation**:
  - Use convolution with Sobel (or another gradient mask) to compute horizontal (Gx) and vertical (Gy)) gradients.
- **Gradient Magnitude**:
  - Calculate the overall gradient magnitude:

        - G (x, y) = sqrt(G(x)^2 + G(y)^2)
- **Apply certain thresholds to retain significant edges**:
    - E(x,y) = 255 if G(x, y) > T or 0 elsewise (T is the threshold))

4.3. Blob detection

4.3.1. Set the [min max] values of the three threshold parameters (highlighted with blue) that you found in the first part of the lab.

4.3.2. Make sure that the manual switch (Enable Switch) is set to zero.

4.3.3. Doubleclick on the Video Compressed Display blocks.

4.3.4. Compile and run the model and look at the video display window.

4.3.5. Bring your colored piece in front of the robot and see if it is completely detected by the robot. Move if back and force, up and down and look at the Blob Centroid display (in blue). If the object is not fully detected as a blob, tune the [min max] threshold values until your object is fully detected as one blob.



Figure 34: Detect object as blob

4.3.6. Explain how the blob centroids could be used to plan the motion of the robot.

The blob centroid, represented by the red dot in the image, serves as a key reference point for motion planning. Here's how it can be used:

- **Identify the Goal Position**:
    - The blob centroid acts as the target or goal position for the robot to reach.
- **Determine the Robot's Current Position**:
    - Knowing the robot's current location relative to the image frame (e.g., the bottom row of the image corresponds to the robot's position).
- **Compute the Vector to the Goal**:
    - Calculate the difference between the robot's current position and the blob centroid.

    - This vector provides both the direction and distance the robot needs to travel.
- **Generate Motion Commands**:
    - Use control algorithms (e.g., PID control, proportional control) to translate the vector into motor commands.
    - Adjust the robot's speed and steering to move toward the goal.
- **Implement Real-Time Feedback**:
    - Continuously update the robot's position and the blob centroid.
    - Adapt the motion commands as needed to ensure accurate tracking and convergence to the goal.
- **Handle Dynamic Scenarios**:
    - If the object (blob) moves, the centroid will change dynamically.
    - The robot can track the moving object by continuously recalculating the vector to the updated centroid.
- **Example Scenario**:
    - Suppose the robot is tasked with following a line or approaching an object:
    - The blob centroid represents the line or object's position.
    - The robot uses the centroid to steer itself along the desired path, adjusting its trajectory as needed.

4.4. Reasoning and Motion Planning

4.4.1. Open the supplied model, QBot2e_Image_Proc_Line_Following.mdl, and make sure the enable switch (Manual Switch) is set to zero. Compile the model and run it.

4.4.2. Doubleclick on the two Video Compressed Display blocks in your model.

4.4.3. Pick a color tape of your choice, and tape in the floor to make a line for the robot to follow as shown in .

4.4.4. Put the robot on the floor right in front of your start point of the line. Tune the [min max] values of the three threshold values, th_red, th_green and th_blue, highlighted with blue, while looking at the Video Compressed Display window so that the line is clearly filtered in the resulting image. To achieve good results, move your cursor over different points in the video display and use the RGB information in the title to estimate the RGB range of values for this card. Once the tuning is successful, enable the manual switch and see if the robot can follow the line. You can change the Kv gain (in

blue), which is the control gain of the robot. The larger this value, the faster the robot will move. Note that by increasing Kv, you might make the controller unstable. QBOT 2E Workbook Student v 1.0

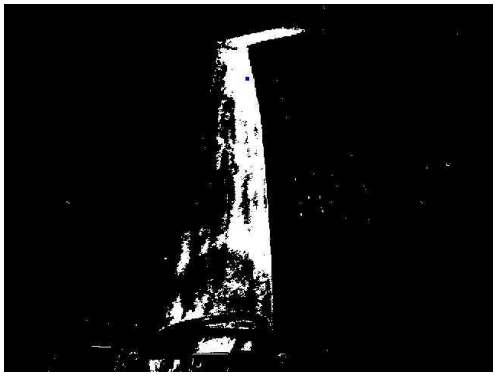


Figure 35: Blob detection for path planning

The blue dot near the top of the vertical structure suggests that this is the centroid of a detected blob. This indicates that the robot's vision system has successfully identified and localized the object. During real-time testing, the robot successfully follows the colored line.

5. Lab 5

Q.5. As you follow the instructions above, please document your own data (graphs, figures, etc) at each step and explain thoroughly how the process of mapping using Occupancy Grid Map, path planning using A* algorithm, and point tracking controller work in driving a robot from a start to end point?

Step 1: Occupancy Grid Mapping

Objective:

To build a representation of the environment where the robot can identify free space and obstacles.

Process:

- The QBot 2e equipped with a Kinect sensor was manually driven around an obstacle (a box placed ~1 m in front of it) using keyboard controls.
- As the robot moved, depth data from the Kinect was processed into a 2D occupancy grid map, which represents the environment as a grid of cells:
    - 0: Free space
    - 1: Obstacle
    - Unknown: Unexplored regions

Example:

After completing the exploration, a map like the one below was generated:

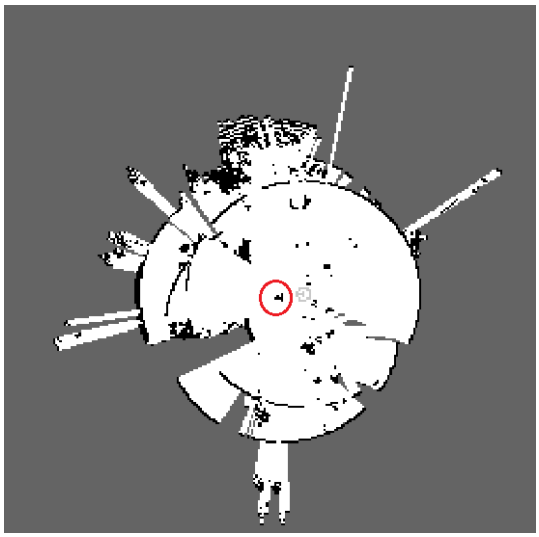


Figure 36 Configuration of the robot and obstacle for occupancy grid mapping required for path planning. (The object is in red circle)

Step 2: Post-processing Steps: Using the **process_map.m** script, the raw map was refined through the following steps:

1. Resizing – To reduce computation time by focusing only on the relevant workspace.
2. Noise Filtering – Removing small unoccupied pixels that were likely due to sensor noise.
3. Padding – Adding a safety margin around the map edges.
4. Safe Zone Expansion – Expanding occupied zones to account for the robot's physical size (radius = 18 cm), ensuring safe clearance between the robot and obstacles.
5. Setting Start Location – Clicking to define the initial pose of the robot on the map. The resulting processed map (**map_extracted.mat**) was then used for path planning.

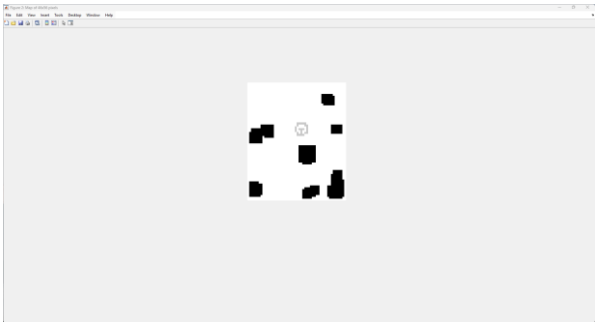


Figure 37: Safe zone embedded image

Step 3: A* Algorithm for Path Planning

Objective:To compute the most optimal collision-free path from the robot's current position to a user-defined goal location.

Algorithm Overview: A* is a heuristic search algorithm that evaluates possible paths by calculating a cost function:

$$f(n)=g(n)+h(n)$$

- Where:

- g(n): Actual cost from the start node to current node.
- h(n): Heuristic estimate of the cost from current node to the goal (usually Euclidean or Manhattan distance).
- f(n): Total estimated cost.

Implementation: Using the script astar_path_planning_8dir.m, the robot's start and goal positions were selected by clicking on the map. The A* algorithm computed an optimal path considering eight directional movements (up/down/left/right + diagonals).

Step 4: Point Tracking Controller

Objective: To guide the robot along the planned waypoints accurately and reach the final goal.

Control Strategy:

- The QBot2e_Path_Planning_Motion_Control.mdl model implemented a point-tracking controller.
- This controller uses a PID-based approach to steer the robot towards each waypoint sequentially.
- It computes velocity and angular velocity commands based on the robot's current position and orientation relative to the desired waypoint.

Controller Operation:

- The robot starts at the initial position.
- At each time step, the controller calculates the error in position and heading.
- Based on this error, control inputs (linear and angular velocities) are adjusted to move the robot toward the next waypoint.
- Once a waypoint is reached (within a defined tolerance), the controller moves to the next one until the final goal is achieved.

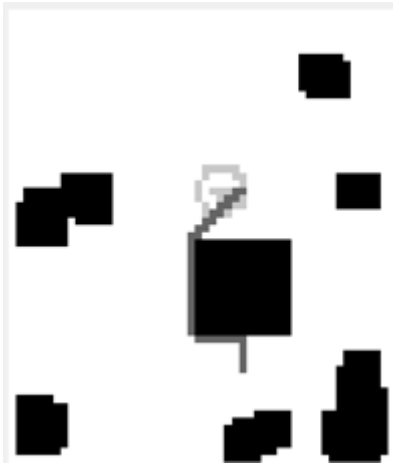


Figure 38: Final path of robot